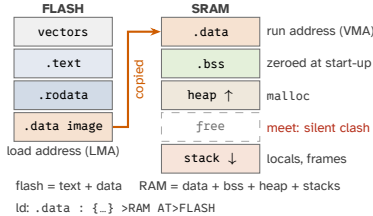
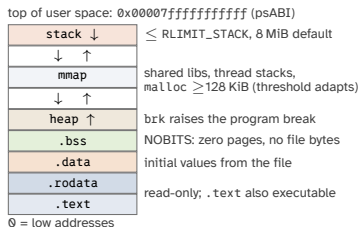


In one line: Four **storage durations**, which the toolchain maps to **sections**: code and constants in `.text`/`.rodata` (flash on an MCU), initialised statics in `.data` (an image copied to RAM at start-up), zero statics in `.bss` (zeroed, no image bytes), locals on the **stack**, `malloc` on the **heap**. The **ABI** fixes sizes and alignments; members sit in order at multiples of their alignment, so structs get **padding**. Byte order is the CPU's; arithmetic below `int` happens **in int**.

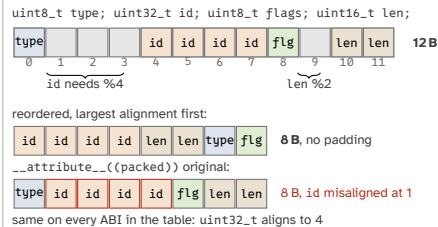
MCU: flash + SRAM, no MMU



Linux x86-64 process (LP64)



struct Msg: 12B, 4 of them padding



Where an object lives

declaration (scope)	duration	section	starts as
<code>int g = 5; (file)</code>	static	<code>.data</code>	5, from the image
<code>int z; (file) static int n; (block)</code>	static	<code>.bss</code>	0 - null - padding 0
<code>const int k = 5; (file)</code>	static	<code>.rodata</code>	5, read-only
<code>"abc"</code>	static	<code>.rodata</code>	write = UB; may be shared
<code>thread_local int t = 1;</code>	thread	<code>.tdat/.tbss</code>	image copied per thread
<code>int x; (block)</code>	automatic	stack / reg.	indeterminate
<code>const int c = 5; (block)</code>	automatic	stack	const ≠ ROM
<code>(int[]){1, 2} (block)</code>	automatic	stack	compound literal
<code>p = malloc(n)</code>	allocated	heap	indeterminate; calloc: 0

C23 defines only the four durations; section names are the ELF / GCC convention. Static and thread objects are initialised before `main` / the thread starts — `int z`; and `int z = 0`; both land in `.bss`.

Before main

- Start-up code (`__crt0`, reset handler) copies `.data` from its **load address** (LMA, flash) to its **run address** (VMA, RAM), zeroes `.bss`, then calls `main`. A hosted loader maps the file instead.
- size `app.elf` (Berkeley): `text` = code *and* read-only data.
- `int x`; at file scope in two files: GCC 10+ and Clang 11+ default to `-fno-common` — a multiple-definition link error (`-fcommon` used to merge them). Header: `extern int x`; one `.c` defines it.

ESP32 (ESP-IDF 6.1)

- Code runs from flash through the MMU cache; `IRAM_ATTR` puts a function in internal IRAM (ISRs flagged `ESP_INTR_FLAG_IRAM`, timing-critical code). `.rodata` goes to DROM (flash, cached); `DRAM_ATTR` forces data into DRAM. IRAM takes 4-byte-aligned word accesses only.
- DRAM holds `.data`, `.bss`, `heap` and `stack`; static data is capped at ~160 KB — the rest of DRAM is `heap` only.
- `__NOINIT_ATTR` → `.noinit`: not initialised at start-up, survives a software restart. RTC slow memory survives deep sleep.

Stack overflow

- No MMU / MPU: nothing faults — the stack runs into the heap or the next task's stack. Causes: recursion, a big local (`uint8_t buf[8192]`), a VLA.
- FreeRTOS: `xTaskCreate` allocates TCB + stack from the heap, `xTaskCreateStatic` takes your buffers. Stack depth and `uxTaskGetStackHighWaterMark()` are in **words** — in **bytes** on ESP-IDF.
- `configCHECK_FOR_STACK_OVERFLOW 1`: stack pointer checked at each switch; 2: the stack is pre-filled with a known value and the last 16 bytes are checked too → `vApplicationStackOverflowHook`. Neither catches every overflow.
- ESP-IDF adds a watchpoint on the last 32 bytes of the running task's stack (`CONFIG_FREERTOS_WATCHPOINT_END_OF_STACK`) and `-fstack-protector` (`CONFIG_COMPILER_STACK_CHECK_MODE`).

Packed and misaligned access

`__attribute__((packed))` sets alignment 1 and drops padding; `m.id` then compiles to byte-wise loads, but `uint32_t *q = &m.id` loses that and `*q` is a misaligned load — GCC 9+ warns by default (`-Waddress-of-packed-member`). `aligned(N)` only raises alignment unless packed is given too.

x86, x86-64 Cortex-M3/M4/M7	works, slower; <code>__m128/256/512</code> operands must be aligned LDR/STR/LDRH/STRH work (trap only with CCR.UNALIGN_TRP); LDRD/STRD/LDM/STM always UsageFault — a double or <code>uint64_t</code> read through a byte pointer
Cortex-M0 ESP32 (Xtensa)	no unaligned access at all: HardFault LoadStoreAlignment panic: 32-bit only at %4, 16-bit at %2

Where the CPU copes, C still does not: `*(uint32_t *)(&buf + 1)` is UB (alignment and effective type) — the optimiser may emit an aligned-only instruction.

Size and alignment — six ABIs

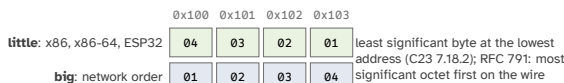
	i386	Arm32	x86-64	AArch64	Apple arm64	Win x64
model	ILP32	ILP32	LP64	LP64	LP64	LLP64
char	signed	unsigned	signed	unsigned	signed	signed
long	4	4	8	8	8	4
long long	8/4	8	8	8	8	8
pointer	4	4	8	8	8	8
double	8/4	8	8	8	8	8
long double	12/4 x87	8	16 x87	16 quad	8	8
SP at a call	16	8	16	16	16	16

a/b = size / alignment where they differ; x87 `long double` uses 10 bytes. Red zone (below SP, usable without moving it): 128 B on x86-64 SysV and Apple arm64, none on Win x64. `{char c; double d;}`: 12 B on i386, 16 elsewhere; `{char c; long l;}`: 16 on LP64, 8 on LLP64 — never `memcpy` a struct across ABIs.

Struct layout — the rules

- C23: members at increasing addresses in declaration order; **no padding before the first** (a struct pointer converts to a pointer to its first member); padding between members and at the end.
- ABIs: each member at the lowest offset that is a multiple of its alignment; struct alignment = its strictest member's; size rounded up to it, so `T a[n]` has no gaps.
- Pin a wire or ABI layout: `static_assert(sizeof(struct Msg) == 12);` `static_assert(offsetof(struct Msg, len) == 10);` — C23 keyword, message optional; `offsetof` is a `size_t` constant (`<stddef.h>`), UB on a bit-field.
- `alignas(64)`, `alignof(T)`: C23 keywords (`<stdalign.h>` is empty); every alignment is a power of two; `max_align_t` has the largest fundamental one. `malloc` (glibc, MSVC): 16-aligned on 64-bit; more → `aligned_alloc`.
- Flexible array member `struct pkt { uint16_t len; uint8_t data[]; };` `sizeof` leaves data out; allocate `sizeof(struct pkt) + n`.
- Bit-fields: storage unit, straddling and bit order are implementation-defined (x86-64: right to left); plain `int` bit-field signedness too; no &. Not for wire formats. C23 `enum op : uint8_t {...}` is 1 byte; a plain `enum`'s type is the compiler's choice.
- Padding bytes**: a store leaves them unspecified — `memcpy` on structs lies, a sent struct leaks old bytes. Statics without an initialiser and C23 = {} zero the padding; otherwise `memset` first.

Byte order: 0x01020304 at 0x100



```
uint32_t rd_be32(const uint8_t *p) { // any alignment, any host
    return (uint32_t)p[0] << 24 | (uint32_t)p[1] << 16
        | (uint32_t)p[2] << 8 | p[3]; } // p[0] << 24 alone is int
uint32_t v; memcpy(&v, buf + 1, 4); v = ntohl(v); // same result
#if __STDC_ENDIAN_NATIVE__ == __STDC_ENDIAN_LITTLE__ // C23 <stdint.h>
```

Shifts act on values: no byte-order dependence. AAPCS64 allows either order. GCC `scalar_storage_order("big-endian")` stores a struct's scalars big-endian (& of such a field is an error).

Integer promotion + usual conversions

- Rank below `int` (`char`, `short`, `uint8_t`, `uint16_t`, `bool`, narrow bit-fields) → `int` if it holds every value, else `unsigned` — before arithmetic, unary + - ~, shifts. `_BitInt(N)` is exempt.
- `uint8_t a = 200, b = 100; a + b` is `int 300`; only `uint8_t r = a + b;` wraps to 44. As `unsigned _BitInt(8), a + b` is 44.
- `~x` for `uint8_t x = 0xFF` is `int -256`, not 0: test `(uint8_t)~x. uint16_t a = 65535; a * a` overflows `int`: UB; write `(uint32_t)a * a. p[0] << 24` with `p[0] >= 0x80`: UB, cast first.
- Mixed signs: `unsigned` of rank $\geq$ the signed wins — `-1 < 1u` and `i < sizeof buf (i = -1)` are **false**; a wider signed type that holds every initialised value wins — `-1L < 1u` is true on LP64, false on ILP32 / LLP64.
- `char c = 0xFF; c == 0xFF` is true where `char` is `unsigned` (Arm32, AArch64 Linux), false where signed: 255 does not fit, GCC wraps it to `-1`.